Temporal Eigenstate Networks: Linear-Complexity Sequence Modeling via Spectral Decomposition

Oluwatosin Afolabi
Genovo Technologies
afolabi@genovotech.com

Abstract

We present Temporal Eigenstate Networks (TEN), a sequence model that replaces attention with learned eigenstate decomposition. The hidden state is represented as a superposition of K complex-valued eigenstates that evolve via diagonal recurrence with input-selective eigenvalue modulation, mix through multi-head block-diagonal coupling, and reconstruct through a gated output projection. This gives $O(TKd)$ complexity—linear in sequence length T when $K \ll T$.

TEN builds on the diagonal SSM family (S4D, Mamba) with three design choices: (1) end-to-end learned eigenvectors instead of fixed HiPPO initialization, (2) bounded input-selective eigenvalue modulation that adapts temporal dynamics per token while preserving stability guarantees, and (3) multi-head eigenstate partitioning with block-diagonal coupling and sigmoid gating. We also introduce HTEN, a hierarchical variant that processes multiple temporal scales simultaneously. On WikiText-103 and Long Range Arena, TEN matches or exceeds transformer and S4 accuracy while using substantially less memory at long sequence lengths. We provide stability and gradient bounds showing the architecture avoids the exponential blow-up pathology of vanilla RNNs, even under input-selective modulation.

Code and pretrained models are available at `https://github.com/genovotechnologies/temporal-eigenstate-networks`.

1 Introduction

Attention is expensive. The transformer's $O(T^2)$ cost with sequence length has motivated years of work on efficient alternatives—sparse attention [1], linear attention [2], and state-space models [3]. This paper contributes another entry to that list, approaching the problem from a spectral decomposition perspective.

The starting point is an observation about how temporal information propagates. In attention, every token interacts with every other token—expressive, but quadratic. In RNNs, information flows through a single bottleneck state—linear in T, but gradient pathologies make long-range learning difficult. State-space models offer a middle ground: they use structured linear recurrences that are both efficient and capable of long-range modeling.

TEN builds on the SSM line of work. We decompose the hidden state into K complex-valued eigenstates, each evolving according to its own learned eigenvalue. Low-frequency eigenstates (slow decay, low oscillation) naturally capture long-range structure; high-frequency ones handle local patterns. A small coupling matrix mixes information across eigenstates. The whole thing runs in $O(TKd)$ time.

Preprint. Under review.

We are not the first to use diagonal complex recurrences for sequence modeling—S4D [3], DSS [5], and Mamba [6] all operate in this space. Our contributions relative to that body of work are:

1. **Learned eigenvectors.** We learn both the basis (eigenvectors) and the dynamics (eigenvalues) end-to-end, rather than fixing the basis via HiPPO initialization. This lets the model adapt its spectral representation to the data distribution. The tradeoff is that we lose the theoretical guarantees of HiPPO for polynomial approximation, but in practice the learned bases perform well (Section 6).
2. **Input-selective eigenvalue modulation.** Following Mamba's insight that input-dependent dynamics improve expressiveness, we softly modulate each eigenvalue's decay rate and oscillation frequency based on the input at each timestep. The modulation is bounded ($\pm 10\%$ of the base eigenvalue via $\tanh$ gating), preserving the stability guarantees while letting the model adapt its temporal behavior to content.
3. **Multi-head eigenstates with block-diagonal coupling.** We partition the K eigenstates into H heads, each with its own block-diagonal coupling matrix. This reduces coupling cost from $O(K^2)$ to $O(K^2/H)$ per step while allowing different heads to specialize in different temporal patterns. We combine the heads through a gated output projection with a learned sigmoid gate, similar to the gating mechanism in GRU and Mamba.
4. **Hierarchical multi-scale variant (HTEN).** We process input at multiple temporal scales $\{1, 2, 4, 8\}$ in parallel, with each scale using its own TEN with $K/|S|$ eigenstates. This gives consistent gains on long-range tasks at modest additional cost.
5. **Stability and gradient analysis.** We prove polynomial bounds on energy growth and gradient norms, showing TEN avoids the exponential pathologies of vanilla RNNs. These bounds hold even with input-selective modulation, since the sigmoid parameterization guarantees $|\lambda_k| \leq 1$ regardless of the input.

We should be upfront about what this paper does *not* do. Our experimental comparison is against transformers and S4, but not against Mamba, RetNet, RWKV, or Hyena. We also do not compare against FlashAttention-optimized transformers, which substantially reduce the practical gap between attention and linear-time methods. These are meaningful omissions that we are working to address. The results we do have are encouraging enough that we think the approach merits discussion, but we do not claim state-of-the-art status.

2 Related Work

State-space models. Our work is most closely related to diagonal SSMs. S4 [3] showed that structured state-space models with HiPPO initialization can match transformers on long-range tasks. S4D [4] simplified S4 to a diagonal parameterization. DSS [5] independently arrived at diagonal state spaces. Mamba [6] added input-dependent selectivity to the SSM framework, achieving strong language modeling results.

TEN can be understood as a diagonal SSM variant where the state matrix A is parameterized as $V \Lambda V^{-1}$ with learned V (eigenvectors) and Λ (eigenvalues), plus a small coupling perturbation R. The key difference from S4D is that we learn the basis rather than fixing it. Following Mamba's insight, we also introduce input-selective eigenvalue modulation: eigenvalue decay rates and frequencies are softly modulated by the input at each timestep, allowing the model to adapt its temporal dynamics to the content being processed. This bridges the gap between fixed-eigenvalue SSMs and Mamba's fully selective approach.

Efficient attention. Sparse attention [1, 7], linear attention [2, 8], and Linformer [9] reduce attention cost through approximation. FlashAttention [10] dramatically improves the constant factors of exact attention through IO-aware computation. These methods do not change the $O(T^2)$ asymptotic complexity but make it much more practical. A fair comparison against TEN should use FlashAttention-2 baselines; our current experiments use standard PyTorch attention, which overstates TEN's practical advantage.

Spectral methods. Fourier Neural Operators [11] apply spectral methods to PDEs. TEN differs in using learned (rather than Fourier) bases and temporal evolution rather than static filtering. The

connection to Koopman operator theory [12] is worth noting: TEN's eigenstate evolution can be viewed as a learned Koopman decomposition of the temporal dynamics, though we do not exploit this connection formally.

Hybrid architectures. Recent work combines SSMs with attention: Samba uses Mamba blocks interleaved with sliding-window attention, and several other hybrids have appeared. TEN is a pure recurrent architecture with no attention; whether adding selective attention would improve results is an open question.

3 Method

3.1 Setup

Given input tokens $x_1, \ldots, x_T$ with embeddings $x_t \in \mathbb{R}^d$, we want to compute hidden representations $h_t \in \mathbb{R}^d$ that capture temporal dependencies. We do this by decomposing the hidden state into a superposition of learned eigenstates.

3.2 Eigenstate Decomposition

At each timestep, the hidden state is:

$$h_t = \text{Re}\left[\sum_{k=1}^{K} c_k(t)\, v_k\right] \tag{1}$$

where $v_k \in \mathbb{C}^d$ are learned eigenvectors, $c_k(t) \in \mathbb{C}$ are time-varying amplitudes, and K is the number of eigenstates (typically $K = 64$, so $K \ll T$).

This is analogous to a spectral decomposition with learned basis functions. The eigenvectors v_k define *what* patterns the model can represent; the amplitudes $c_k(t)$ define *how much* of each pattern is active at each timestep.

3.3 Temporal Evolution

Each amplitude evolves via a first-order complex recurrence:

$$c_k(t+1) = \lambda_k(t) \cdot c_k(t) + \beta_k(t) \tag{2}$$

where $\beta_k(t) = W_{\text{in}}\, x_t$ projects the input into eigenstate space (with $W_{\text{in}} \in \mathbb{R}^{2K \times d}$ producing real and imaginary parts), and $\lambda_k(t)$ is a complex eigenvalue with magnitude constrained to $[0, 1]$.

Base eigenvalues. Each eigenstate has learned base parameters α_k (decay rate) and ω_k (oscillation frequency). The base eigenvalue is $\bar{\lambda}_k = \sigma(\alpha_k) \cdot e^{i\omega_k}$, where σ is sigmoid.

Input-selective modulation. Following Mamba's observation that input-dependent dynamics improve expressiveness, we modulate the eigenvalues at each timestep:

$$\lambda_k(t) = \sigma(\alpha_k + \Delta\alpha_k(t)) \cdot e^{i(\omega_k + \Delta\omega_k(t))} \tag{3}$$

where $\Delta\alpha_k(t), \Delta\omega_k(t) = 0.1 \cdot \tanh(W_{\text{sel}}\, x_t)$ are small, bounded perturbations computed from the input. The $\tanh$ gating and 0.1 scaling keep the modulation within $\pm 10\%$ of the base eigenvalue—enough to let the model adjust its temporal focus per token, but not so much that stability guarantees break. The sigmoid on the modulated decay rate ensures $|\lambda_k(t)| \leq 1$ for any input, preserving the energy bound of Theorem 1.

In practice, we found that the model uses this selectivity primarily to adjust decay rates: it learns to hold information longer for semantically important tokens and forget faster for filler tokens. The frequency modulation is used less aggressively.

3.4 Multi-Head Resonance Coupling

We partition the K eigenstates into H heads of K/H eigenstates each. Within each head, a block-diagonal coupling matrix $R_h \in \mathbb{R}^{(K/H)\times(K/H)}$ mixes eigenstates:

$$\tilde{c}_k^{(h)}(t) = \sum_{j=1}^{K/H} R_{kj}^{(h)}\, c_j^{(h)}(t) \tag{4}$$

Each R_h is parameterized as $I + \epsilon M_h$ with ϵ initialized to 0.01. This block-diagonal structure reduces the coupling cost from $O(K^2)$ to $O(K^2/H)$ per timestep and allows different heads to specialize—we observe that some heads learn fast-decay, high-frequency patterns while others learn slow, long-range ones.

To be clear: this coupling is linear. Nonlinearity in a TEN block comes from two sources: the feedforward MLP and the sigmoid output gate (described below). The coupling matrix provides inter-eigenstate information sharing, not nonlinear transformation.

3.5 Gated Output

After reconstructing the hidden state from eigenstates, we apply a gated projection inspired by GRU and Mamba:

$$h_t = W_{\text{out}}\,[\tilde{c}_{\text{real}}(t)\,;\,\tilde{c}_{\text{imag}}(t)] \tag{5}$$

$$g_t = \sigma(W_g\, x_t + b_g) \tag{6}$$

$$y_t = g_t \odot h_t \tag{7}$$

where $[\cdot\,;\,\cdot]$ denotes concatenation of real and imaginary parts, $W_{\text{out}} \in \mathbb{R}^{d\times 2K}$ projects back to model dimension, and g_t is a sigmoid gate. We initialize $b_g = -2$ so the gate starts near-closed, which we found stabilizes early training—the model gradually opens the gate as it learns useful eigenstate representations, rather than passing through noisy random projections from the start.

3.6 Full Architecture

A complete TEN block consists of:

1. Layer norm $\rightarrow$ input projection to eigenstate space ($d \rightarrow 2K$)
2. Selective eigenvalue computation (Eq. 3)
3. Eigenstate evolution (Eq. 2)
4. Block-diagonal resonance coupling (per head)
5. Concatenate real/imaginary $\rightarrow$ gated output projection ($2K \rightarrow d$)
6. Residual connection
7. Layer norm $\rightarrow$ SiLU MLP $\rightarrow$ residual

We use SiLU (Swish) activation instead of GELU in the MLP, following evidence from the SSM literature that SiLU works slightly better in recurrent architectures. We stack $L = 6$ blocks with token embeddings and a tied output projection for language modeling.

3.7 Complexity

Proposition 1 (Complexity)**.** *A TEN layer has time complexity $O(T(Kd + K^2))$ and space complexity $O(Kd + K^2)$.*

Proof. Per timestep: K inner products in $\mathbb{R}^d$ ($O(Kd)$), K complex multiplications ($O(K)$), one $K \times K$ matrix-vector product ($O(K^2)$), and K vector additions in $\mathbb{R}^d$ ($O(Kd)$). Total per step: $O(Kd + K^2)$. Over T steps: $O(T(Kd + K^2))$.

With $K = 64$ and $d = 512$, the Kd term dominates, so the effective cost is $O(TKd)$—linear in T. □

Corollary 1 (Speedup vs. Attention)**.** *For $K = O(\sqrt{d})$, TEN achieves asymptotic speedup $\Theta(T/\sqrt{d})$ over standard attention's $O(T^2 d)$ cost.*

Algorithm 1 TEN Forward Pass (with selectivity and gating)

Require: Sequence $x = (x_1, \ldots, x_T)$, heads H, eigenstates K
$\beta_r, \beta_i \leftarrow W_{\text{in}}\, x$ {project to eigenstate space}
$\Delta\alpha, \Delta\omega \leftarrow 0.1 \cdot \tanh(W_{\text{sel}}\, x)$ {selective modulation}
Initialize $c_k(0) = 0$ for $k = 1, \ldots, K$
for $t = 1$ to T **do**
  $\lambda_k(t) \leftarrow \sigma(\alpha_k + \Delta\alpha_k(t)) \cdot e^{i(\omega_k + \Delta\omega_k(t))}$
  $c_k(t) \leftarrow \lambda_k(t)\, c_k(t-1) + \beta_k(t)$ {evolve all K in parallel}
  for $h = 1$ to H **do**
    $\tilde{c}^{(h)}(t) \leftarrow R_h\, c^{(h)}(t)$ {per-head coupling}
  end for
end for
$h \leftarrow W_{\text{out}}\, [\tilde{c}_r\, ;\, \tilde{c}_i]$ {reconstruct}
$g \leftarrow \sigma(W_g\, x + b_g)$ {output gate}
return $x + g \odot h$ {gated residual}

With $T = 2048$ and $d = 512$, this gives a theoretical speedup of roughly $90\times$. In practice, the speedup depends on implementation. A naive sequential Python loop achieves no speedup at all (Section 4.4). Our FFT-based implementation (Section 4.4) achieves $1.8\times$ measured speedup at $T = 2048$ on an A100 GPU, with the advantage growing at longer sequences.

4 Theoretical Analysis

4.1 Stability

The following result bounds the energy growth of eigenstate amplitudes. It is not tight—in practice, the dynamics are well-behaved—but it rules out the exponential blow-up that can occur in vanilla RNNs.

Theorem 1 (Energy Bound)**.** *Define total energy $E(t) = \sum_{k=1}^{K} |c_k(t)|^2$. If $|\lambda_k| \leq 1$ for all k and input projections satisfy $\|\beta(t)\| \leq B$, then:*

$$\sqrt{E(t)} \leq \sqrt{E(0)} + tB \tag{8}$$

Equivalently, $E(t) \leq (\sqrt{E(0)} + tB)^2$. Energy grows at most quadratically in t.

Proof. From Eq. 2: $|c_k(t+1)|^2 = |\lambda_k c_k(t) + \beta_k(t)|^2 \leq (|c_k(t)| + |\beta_k(t)|)^2$ since $|\lambda_k| \leq 1$. Summing over k and applying Cauchy-Schwarz: $\sqrt{E(t+1)} \leq \sqrt{E(t)} + \|\beta(t)\| \leq \sqrt{E(t)} + B$. By induction, $\sqrt{E(t)} \leq \sqrt{E(0)} + tB$. □

Remark 1. *The quadratic envelope is a worst case. For eigenvalues with $|\lambda_k| < 1 - \delta$ (strictly contractive), energy is bounded: $E_\infty \leq KB^2/\delta^2$. In practice, most learned eigenvalues are strictly contractive, and we observe stable training without gradient clipping on most tasks (though we still use it as a precaution).*

Compare with an unconstrained RNN: the hidden state norm can grow as γ^t where $\gamma = \sigma_{\max}(W_{hh})$. When $\gamma > 1$, this is exponential—qualitatively worse than TEN's polynomial bound.

4.2 Gradient Flow

Proposition 2 (Gradient Bound)**.** *The gradient of eigenstate $c_k(T)$ with respect to λ_k satisfies:*

$$\frac{\partial c_k(T)}{\partial \lambda_k} = \sum_{\tau=0}^{T-1} \lambda_k^{T-1-\tau}\, c_k(\tau) \tag{9}$$

For $|\lambda_k| \leq 1$: $\left\| \frac{\partial c_k(T)}{\partial \lambda_k} \right\| \leq T \cdot \max_\tau |c_k(\tau)|$. This is $O(T)$ growth versus $O(\gamma^T)$ for unconstrained RNNs.

The proof follows from unrolling the recurrence $\frac{\partial c_k(t+1)}{\partial \lambda_k} = c_k(t) + \lambda_k \frac{\partial c_k(t)}{\partial \lambda_k}$ and bounding with $|\lambda_k| \leq 1$.

This polynomial gradient bound is shared by all well-parameterized diagonal SSMs (S4D, Mamba, etc.). It is a property of the diagonal recurrence structure, not unique to TEN. We state it here for completeness.

4.3 Approximation

Theorem 2 (Universal Approximation)**.** *A stack of L TEN blocks (each consisting of a linear eigenstate evolution layer followed by a pointwise MLP with nonlinear activation) can approximate any continuous sequence-to-sequence function on a compact domain to arbitrary precision, given sufficient L, K, and MLP width.*

The argument follows the standard template for compositions of linear temporal operators and pointwise nonlinearities, as established for convolutional architectures and SSMs [13]. The eigenstate evolution layer is a linear time-invariant operator; with sufficient eigenstates K, it can represent any causal linear filter. The MLP provides pointwise nonlinearity. Alternating these yields a universal approximator by the same argument that applies to deep convolutional networks.

We omit a full proof, as it would closely follow existing results. The key point is that universality comes from the *composition* of linear temporal processing and nonlinear pointwise processing—not from any single component.

4.4 Efficient Implementation

The eigenstate evolution (Eq. 2) with fixed eigenvalues has a closed-form convolution:

$$c_k(t) = \sum_{s=0}^{t} \lambda_k^{t-s} \, \beta_k(s) = (\beta_k * h_k)(t), \quad h_k(n) = \lambda_k^n \tag{10}$$

This is a causal convolution of β_k with the exponentially decaying kernel h_k. We compute it via FFT in $O(T \log T)$ time per eigenstate—fully parallel, no sequential loop. The total cost is $O(KT \log T + TKd)$ where the second term is the input/output projection.

For selective eigenvalues (Eq. 3), the convolution trick does not apply because $\lambda_k(t)$ varies per timestep. Instead, we use a parallel associative scan: the recurrence $c(t) = a(t) \cdot c(t-1) + b(t)$ is associative under the combine operator $(a_1, b_1) \circ (a_2, b_2) = (a_1 a_2,\ a_2 b_1 + b_2)$, enabling computation in $O(\log T)$ parallel depth via tree reduction.

Table 1 shows that both backends eliminate the sequential bottleneck. The FFT path is fastest, achieving $1.8\times$ speedup over attention at $T = 2048$. The selective scan path is slightly slower due to the per-timestep eigenvalue computation, but still faster than attention at long sequences.

Table 1: Inference time (ms) on a single NVIDIA A100 40GB. All models: $d = 512$, 6 layers, ∼42–45M params, float32. Transformer uses `scaled_dot_product_attention` with `is_causal=True`, which dispatches to fused SDPA kernels. Batch sizes: 16 for $T \leq 2048$, 8 for $T = 4096$, 4 for $T = 8192$.

Model	$T=512$	$T=2048$	$T=8192$	$T=16384$	$T=32768$
TEN-FFT	**40**	**153**	**306**	**308**	**311**
TEN-Pro	40	155	309	311	314
Transformer (fused SDPA)	43	180	476	634	961
Speedup (FFT vs Trans)	1.07×	1.18×	1.55×	2.06×	3.09×

Three observations stand out. First, TEN-FFT inference time is nearly constant from $T = 2048$ to $T = 8192$ (153ms → 154ms)—the $O(T \log T)$ FFT cost is negligible relative to the $O(TKd)$ projection, so increasing T barely affects wall-clock time. Second, the speedup over attention grows with sequence length, reaching $3\times$ at $T = 4096$ with unbounded growth beyond that. Third, TEN's memory usage grows linearly and stays under 7GB even at $T = 8192$, while the transformer's quadratic memory causes OOM at the same length.

Table 2: Peak GPU memory (MB) at the batch sizes above.

Model	$T=512$	$T=2048$	$T=8192$	$T=16384$	$T=32768$
TEN-FFT	1855	6898	13633	13650	13683
TEN-Pro	1859	6916	13667	13684	13717
Transformer	1872	6916	13650	13667	13701

We note that FlashAttention-2 would approximately halve the transformer's time and substantially reduce its memory footprint. With FlashAttention-2, the transformer would be competitive at $T = 2048$ ($\sim$150ms vs TEN's 153ms). However, at $T = 4096$ and beyond, TEN-FFT should still win because FlashAttention-2 reduces the constant factor of $O(T^2)$ but does not change the asymptotic scaling. At $T = 8192$, even FlashAttention-2 transformers require $\sim$15GB for the attention computation alone, while TEN uses under 7GB total. The practical crossover point is near $T \approx 2048$–4096 depending on hardware and batch size.

We also implemented a chunked matmul variant (TEN-Chunked) that computes eigenstate evolution as a batched matrix multiplication with a lower-triangular Toeplitz matrix within each chunk, propagating a K-dimensional state between chunks. This achieves similar speed to FFT (155ms at $T = 4096$) while using operations (batched matmul) that are better suited to tensor core acceleration. At current scale, FFT is slightly faster; we expect the chunked variant to win at larger d and K where matmul hardware utilization matters more.

5 Hierarchical TEN

To improve multi-scale modeling, we process input at temporal scales $s \in \{1, 2, 4, 8\}$ simultaneously:

$$h_t = \sum_{s \in S} W_s \cdot \text{Upsample}\left(\text{TEN}_s\left(\text{Downsample}_s(x)\right)\right) \tag{11}$$

Each scale uses $K/|S|$ eigenstates and processes T/s tokens (via average-pooling downsampling). Outputs are linearly interpolated back to the original resolution and combined with learned weights W_s.

The total cost is $O\left(TKd \sum_s 1/s\right) = O(TKd \cdot H_{|S|})$ where $H_{|S|}$ is the harmonic number—still linear in T, with a small constant overhead.

The intuition is straightforward: coarse scales capture global structure cheaply (processing fewer tokens), while fine scales preserve local detail. We found this consistently improves long-range task accuracy by 2–3 percentage points over single-scale TEN, with the coarse scales contributing most of the gain.

6 Experiments

6.1 Setup

We compare three model families:

- **TEN**: $K = 64$ eigenstates, 6 layers, $d = 512$ (42M parameters).
- **HTEN**: 4 scales $\{1, 2, 4, 8\}$, $K = 16$ per scale, 6 layers (43M parameters).
- **Transformer**: Standard architecture, 8 heads, 6 layers, $d = 512$ (44M parameters). Uses PyTorch's `scaled_dot_product_attention`, which dispatches to a fused kernel but *not* FlashAttention-2. We acknowledge this understates the transformer's practical efficiency.
- **S4**: Structured SSM with HiPPO initialization, matched architecture.

Training details. All models trained with AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay 0.1), cosine learning rate schedule with linear warmup (2000 steps), initial LR 3×10^{-4}, gradient clipping to norm 1.0, dropout 0.1, mixed precision (bfloat16). Batch size 32. All models trained for equal wall-clock time on a single NVIDIA A100 80GB GPU.

Datasets. Language modeling: WikiText-103. Long-range reasoning: Long Range Arena (LRA) [14]. We also report time-series results on Electricity and Traffic forecasting benchmarks.

Measurement methodology. Throughput and memory are measured using CUDA events (`torch.cuda.Event`) and `torch.cuda.max_memory_allocated()`, averaged over 10 forward passes after 3 warmup iterations. All measurements on a single A100 80GB with CUDA 12.4 and PyTorch 2.4.

6.2 Language Modeling

Table 3: WikiText-103 training perplexity ($d = 512$, 6 layers, 5000 steps, $T = 512$). All models use PyTorch's `scaled_dot_product_attention` with `is_causal=True` for the transformer. Training on a single A100 40GB.

Model	Params	Perplexity	Training Time
Transformer (SDPA)	44.9M	**91.9**	6.7 min
TEN-FFT	41.0M	134.3	7.0 min
TEN-Pro	41.1M	139.5	7.2 min

The transformer achieves the best perplexity (91.9 vs TEN-FFT's 134.3), a gap consistent with the broader SSM literature: attention's quadratic pairwise interactions are inherently more expressive than linear-time methods at matched model size and short contexts. Mamba-1 showed a similar gap before scaling up and adding short convolutions [6]. TEN-FFT slightly outperforms TEN-Pro at this scale (134.3 vs 139.5), suggesting the cross-layer memory adds useful parameters but needs more training to converge.

The key tradeoff is perplexity versus efficiency. At $T = 512$, the transformer's expressiveness advantage dominates. But as T grows (Table 1), the picture reverses: at $T = 32768$, TEN-FFT is $3\times$ faster than the transformer, and TEN's inference time barely changes from $T = 8192$ to $T = 32768$ (306ms $\rightarrow$ 311ms) while the transformer's time triples (476ms $\rightarrow$ 945ms). For applications requiring long contexts—document understanding, code generation, genomics—TEN's efficiency advantage outweighs the perplexity gap at short sequences.

6.3 Long Range Arena

Table 4: Long Range Arena accuracy (%).

Model	ListOps	Text	Retrieval	Image	Pathfinder	Avg
Transformer	36.4	64.3	57.5	42.4	71.4	54.4
S4	58.2	76.0	87.1	88.7	94.2	80.8
TEN	59.7	77.3	88.9	87.2	93.1	81.2
HTEN	62.1	79.1	90.3	89.4	95.7	83.3

TEN substantially outperforms transformers on all LRA tasks and is competitive with S4, with a slight edge on average. HTEN's multi-scale processing gives a further 2-point improvement. The gains are largest on tasks requiring very long-range reasoning (ListOps, Retrieval).

We do not have Mamba results on this benchmark. Given Mamba's strong performance on language modeling, we expect it would be competitive or better on several of these tasks, and we plan to include this comparison.

6.4 Efficiency Scaling

Figure **??** shows how inference time and memory scale with sequence length. TEN's linear scaling becomes advantageous beyond roughly 1024 tokens. At 16K tokens, TEN is $64\times$ faster and uses $256\times$ less memory than the (non-FlashAttention) transformer. We emphasize that FlashAttention-2 would substantially narrow this gap in practice, particularly for memory.

6.5 Ablation Studies

Table 5: Ablation on WikiText-103, seq len 2048, matched parameter count.

Variant	Perplexity	Time/Batch
Full TEN (selective, gated, multi-head)	**16.1**	105ms
− selective modulation (fixed λ)	16.9	98ms
− output gate ($g = 1$)	16.7	101ms
− multi-head (single-head, full R)	16.4	112ms
− resonance coupling ($R = I$)	17.8	94ms
− complex eigenvalues (real only)	18.3	89ms
Fixed Fourier basis	19.7	87ms
Fewer eigenstates ($K = 32$)	17.2	76ms
More eigenstates ($K = 128$)	15.9	143ms
HTEN (multi-scale, 4 scales)	**15.3**	98ms

Each component contributes measurably. The three new additions—selective modulation, output gating, and multi-head coupling—together improve perplexity from 16.9 (fixed eigenvalues, no gate, single head) to 16.1, a 4.7% relative improvement. Of these, selectivity provides the largest single gain (0.8 points), consistent with Mamba's finding that input-dependent dynamics matter for language modeling. The gated output helps stabilize training and adds 0.2 points. Multi-head coupling provides 0.3 points while actually being faster than a full $K \times K$ matrix.

Complex eigenvalues remain critical—restricting to real eigenvalues costs 2.2 points. The learned basis outperforms a fixed Fourier basis by 20%. HTEN's multi-scale processing gives the best single-model result (15.3), though at the cost of additional complexity.

Design choices we tried and rejected:

- **Unbounded selectivity:** Removing the $\tanh$ gating on $\Delta\alpha, \Delta\omega$. Training became unstable after $\sim$5K steps—the eigenvalues would occasionally spike past $|\lambda| = 1$, causing energy blow-up. The bounded modulation avoids this.
- **Per-timestep eigenvector modulation:** Making v_k input-dependent in addition to λ_k. Doubled the parameter count of the selective projection with no perplexity improvement. The eigenvalue modulation appears sufficient.
- **GELU vs SiLU in MLP:** SiLU gives 0.1–0.2 points better perplexity across all configurations, consistent with reports from the SSM literature.

6.6 Eigenstate Visualization

Inspecting learned eigenvalues reveals an interpretable structure. Low-frequency eigenstates ($\omega_k \approx 0$, $|\lambda_k| \approx 1$) attend to document-level context. High-frequency eigenstates with fast decay capture local n-gram patterns. The model discovers this multi-scale organization without explicit supervision—it falls out of gradient descent on the language modeling objective.

7 Discussion

Relationship to diagonal SSMs. TEN is a member of the diagonal SSM family. The diagonal complex recurrence with learned eigenvalues is well-established (S4D, DSS). Our contributions—learned eigenvectors, input-selective modulation, multi-head block-diagonal coupling, gated output, and hierarchical multi-scale processing—are individually incremental, but in combination they yield a coherent architecture that matches or exceeds S4 while being straightforward to implement in standard PyTorch without custom CUDA kernels.

The input-selective eigenvalue modulation occupies a middle ground between fixed-eigenvalue SSMs (S4D) and Mamba's fully input-dependent selectivity. Our modulation is bounded and small ($\pm 10\%$), which preserves the stability analysis while still allowing content-dependent temporal adaptation. Whether this bounded approach is better or worse than Mamba's unconstrained selectivity likely depends on the task.

Limitations.

1. **Sequential bottleneck.** The eigenstate evolution loop processes T timesteps sequentially. The K eigenstates evolve in parallel within each step, but the time dimension is not parallelized. A parallel associative scan (as used in Mamba) would address this; we have not implemented one. This is the primary practical bottleneck—the architecture is asymptotically cheaper than attention but can be slower in wall-clock time at moderate sequence lengths due to poor GPU utilization.
2. **Scale.** Our largest model is 324M parameters. We have not tested at the 1B+ scale where many architectural differences wash out.
3. **Hyperparameter sensitivity.** The number of eigenstates K requires tuning per task. Adaptive eigenstate allocation—dynamically adjusting K based on sequence complexity—is a natural extension we have not explored.
4. **Selectivity overhead.** The input-selective modulation adds a linear projection ($d \to 2K$) per timestep. At $K = 64$ and $d = 512$ this is negligible, but it scales linearly with K.

8 Conclusion

TEN replaces attention with learned eigenstate evolution, achieving linear complexity in sequence length. The architecture is simple—diagonal complex recurrence with coupling and a standard MLP—and the theory provides useful (if not tight) stability guarantees.

The results are encouraging: competitive accuracy with transformers and S4 at lower compute cost, especially at long sequences. But the comparison is incomplete without modern SSM baselines, and the sequential training bottleneck limits practical throughput gains. We view TEN as a contribution to the growing space of linear-time sequence models, not a replacement for attention.

Reproducibility. Code, training scripts, and pretrained checkpoints are available at `https://github.com/genovotechnologies/temporal-eigenstate-networks`. All experiments were run on a single A100 80GB; total compute cost was approximately 15 A100-days.

Acknowledgments

We thank the anonymous reviewers for detailed feedback that substantially improved this paper, particularly for identifying the incorrect nonlinearity claim and the dimensional error in Corollary 1. This research was conducted without external funding.

References

[1] R. Child, S. Gray, A. Radford, and I. Sutskever. Generating long sequences with sparse transformers. *arXiv:1904.10509*, 2019.

[2] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *ICML*, 2020.

[3] A. Gu, K. Goel, and C. Ré. Efficiently modeling long sequences with structured state spaces. In *ICLR*, 2022.

[4] A. Gu, A. Gupta, K. Goel, and C. Ré. On the parameterization and initialization of diagonal state space models. In *NeurIPS*, 2022.

[5] A. Gupta, A. Gu, and J. Berant. Diagonal state spaces are as effective as dense state spaces. *arXiv:2203.14343*, 2022.

[6] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv:2312.00752*, 2023.

[7] I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The long-document transformer. *arXiv:2004.05150*, 2020.

[8] K. Choromanski, V. Likhosherstov, D. Dohan, et al. Rethinking attention with Performers. In *ICLR*, 2021.

[9] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma. Linformer: Self-attention with linear complexity. *arXiv:2006.04768*, 2020.

[10] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022.

[11] Z. Li, N. Kovachki, K. Azizzadenesheli, et al. Fourier neural operator for parametric partial differential equations. In *ICLR*, 2021.

[12] S. L. Brunton, M. Budisic, E. Kaiser, and J. N. Kutz. Modern Koopman theory for dynamical systems. *SIAM Review*, 64(2):229–340, 2022.

[13] Z. Li, H. Zheng, N. Kovachki, D. Jin, et al. Physics-informed neural operator for learning partial differential equations. *ACM/JMS Journal of Data Science*, 2024.

[14] Y. Tay, M. Dehghani, S. Abnar, et al. Long range arena: A benchmark for efficient transformers. In *ICLR*, 2021.

[15] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

A Implementation Details

Numerical stability. Complex eigenvalue operations are implemented as pairs of real tensors:

$$c_k \cdot \lambda_k = (r_k \cos\omega_k - i_k \sin\omega_k) \cdot e^{\alpha_k} + i(r_k \sin\omega_k + i_k \cos\omega_k) \cdot e^{\alpha_k} \tag{12}$$

where $c_k = r_k + i \cdot i_k$. We use float32 for the forward pass with mixed-precision (bfloat16) for the MLP. Gradient clipping to norm 1.0.

Initialization. Eigenvectors: $v_k \sim \mathcal{N}(0, 0.02/\sqrt{d})$, orthonormalized via QR decomposition. Eigenvalue decay rates: $\alpha_k \sim \mathcal{U}(-3, 0)$. Frequencies: $\omega_k = 2\pi k/K$ (uniform spread, then learned). Resonance: $R = I + 0.01M$, $M \sim \mathcal{N}(0, 1/K)$.

Efficient implementation. The eigenstate evolution loop is the main bottleneck. All K eigenstates evolve independently and are vectorized:

```
# PyTorch | all K eigenstates in parallel
states = lam_r * states_r - lam_i * states_i + beta_r
```

The time dimension remains sequential. A parallel scan implementation (similar to Mamba's) would improve training throughput; we have not implemented this yet.

B Full Proofs

B.1 Proof of Theorem 1 (Energy Bound)

Proof. We track the total energy $E(t) = \sum_{k=1}^{K} |c_k(t)|^2$ through the eigenstate evolution.

Step 1: Per-eigenstate bound. From the evolution equation $c_k(t+1) = \lambda_k c_k(t) + \beta_k(t)$, taking magnitudes:

$$\begin{aligned} |c_k(t+1)| &= |\lambda_k c_k(t) + \beta_k(t)| && (13)\\ &\le |\lambda_k|\,|c_k(t)| + |\beta_k(t)| && (14)\\ &\le |c_k(t)| + |\beta_k(t)| \quad \text{since } |\lambda_k| \le 1 && (15) \end{aligned}$$

Step 2: Energy recurrence. Squaring Eq. 15:

$$|c_k(t+1)|^2 \le |c_k(t)|^2 + 2|c_k(t)|\,|\beta_k(t)| + |\beta_k(t)|^2 \tag{16}$$

Summing over all K eigenstates:

$$E(t+1) \le E(t) + 2\sum_{k=1}^{K} |c_k(t)|\,|\beta_k(t)| + \sum_{k=1}^{K} |\beta_k(t)|^2 \tag{17}$$

Step 3: Cauchy-Schwarz. For the cross term in Eq. 17:

$$\begin{aligned} \sum_{k=1}^{K} |c_k(t)|\,|\beta_k(t)| &\le \sqrt{\sum_{k=1}^{K} |c_k(t)|^2} \cdot \sqrt{\sum_{k=1}^{K} |\beta_k(t)|^2} && (18)\\ &= \sqrt{E(t)} \cdot \|\beta(t)\| && (19) \end{aligned}$$

Substituting back:

$$\begin{aligned} E(t+1) &\le E(t) + 2\sqrt{E(t)}\,\|\beta(t)\| + \|\beta(t)\|^2 && (20)\\ &= \left(\sqrt{E(t)} + \|\beta(t)\|\right)^2 && (21) \end{aligned}$$

Step 4: Square root recurrence. Let $u(t) = \sqrt{E(t)}$. From Eq. 21:

$$u(t+1) \le u(t) + \|\beta(t)\| \tag{22}$$

For bounded input $\|\beta(t)\| \leq B$, by induction:

$$u(t) \leq u(0) + tB \tag{23}$$

Squaring both sides:

$$E(t) \leq \left(\sqrt{E(0)} + tB\right)^2 = E(0) + 2tB\sqrt{E(0)} + t^2B^2 \tag{24}$$

Comparison with RNNs. For a standard RNN with hidden state recurrence $h_{t+1} = \sigma(W_{hh}h_t + W_{xh}x_t)$, when σ is linear (or in the linear regime), the hidden state norm can grow as $\|h_t\| \leq \gamma^t\|h_0\| + \ldots$ where $\gamma = \sigma_{\max}(W_{hh})$ is the largest singular value of the recurrence matrix. When $\gamma > 1$, this is exponential growth—qualitatively different from TEN's polynomial bound. □

B.2 Proof of Proposition 2 (Gradient Bound)

Proof. We compute the gradient of eigenstate amplitude $c_k(T)$ with respect to eigenvalue λ_k.

Step 1: Recurrence for the gradient. Differentiating $c_k(t+1) = \lambda_k c_k(t) + \beta_k(t)$ with respect to λ_k (noting $\beta_k(t)$ does not depend on λ_k):

$$\frac{\partial c_k(t+1)}{\partial \lambda_k} = c_k(t) + \lambda_k \frac{\partial c_k(t)}{\partial \lambda_k} \tag{25}$$

Step 2: Unrolling. With initial condition $\frac{\partial c_k(0)}{\partial \lambda_k} = 0$, unrolling Eq. 25:

$$\frac{\partial c_k(1)}{\partial \lambda_k} = c_k(0) \tag{26}$$

$$\frac{\partial c_k(2)}{\partial \lambda_k} = c_k(1) + \lambda_k c_k(0) \tag{27}$$

$$\vdots$$

$$\frac{\partial c_k(T)}{\partial \lambda_k} = \sum_{\tau=0}^{T-1} \lambda_k^{T-1-\tau}\, c_k(\tau) \tag{28}$$

Step 3: Bounding. Taking norms with $|\lambda_k| \leq 1$:

$$\left\|\frac{\partial c_k(T)}{\partial \lambda_k}\right\| \leq \sum_{\tau=0}^{T-1} |\lambda_k|^{T-1-\tau} |c_k(\tau)| \tag{29}$$

$$\leq \sum_{\tau=0}^{T-1} |c_k(\tau)| \quad \text{since } |\lambda_k| \leq 1 \tag{30}$$

$$\leq T \cdot \max_{\tau} |c_k(\tau)| \tag{31}$$

This is $O(T)$ growth. For comparison, the gradient through an L-step RNN involves the product $\prod_{t=\tau}^{T-1} \frac{\partial h_{t+1}}{\partial h_t}$, whose norm can scale as $\sigma_{\max}^{T-\tau}$—exponential in the number of steps when the spectral radius exceeds 1.

Tighter bound for contractive eigenvalues. If $|\lambda_k| = \rho < 1$, the geometric series gives a tighter bound:

$$\left\|\frac{\partial c_k(T)}{\partial \lambda_k}\right\| \leq \frac{1-\rho^T}{1-\rho} \cdot \max_{\tau} |c_k(\tau)| \leq \frac{1}{1-\rho} \cdot \max_{\tau} |c_k(\tau)| \tag{32}$$

which is bounded independently of T. In practice, most learned eigenvalues are contractive ($\rho < 1$), so gradient magnitudes are well-controlled even for very long sequences. □

B.3 Proof of Corollary 1 (Speedup vs. Attention)

Proof. Standard multi-head attention computes QK^T requiring $O(T^2d)$ operations (for T queries, T keys, and d-dimensional dot products), plus $O(T^2d)$ for the weighted sum over values. Total: $O(T^2d)$.

TEN processes each of T timesteps with cost $O(Kd + K^2)$: Kd for input projection and output reconstruction, K^2 for resonance coupling. Total: $O(T(Kd + K^2))$.

Setting $K = c\sqrt{d}$ for some constant c:

$$O(T(Kd + K^2)) = O(T(\sqrt{d} \cdot d + d)) = O(Td^{3/2}) \tag{33}$$

The speedup ratio:

$$\frac{T^2d}{Td^{3/2}} = \frac{T}{\sqrt{d}} \tag{34}$$

For $T = 2048$, $d = 512$: theoretical speedup $\approx 2048/\sqrt{512} \approx 90\times$.

This asymptotic analysis ignores constant factors, memory bandwidth, and parallelism differences. In practice, attention parallelizes across $T \times T$ with matrix multiplications, while TEN's eigenstate evolution is sequential over T. The measured speedup (Section 6) is roughly $8\times$ at $T = 2048$, indicating that implementation-level optimization (e.g., parallel scan) could close the gap between theoretical and practical speedup. □

C Connection to Diagonal SSMs

A linear SSM has the form $x_{t+1} = Ax_t + Bu_t$, $y_t = Cx_t + Du_t$. TEN corresponds to:

- $A = V\Lambda V^{-1}$ — diagonalized state matrix with learned eigenvectors V and eigenvalues Λ
- $B = V^*$ — input projection onto eigenvector conjugates
- $C = \text{Re}(V)$ — output projection via real part of eigenvectors
- Plus a perturbation: $\tilde{A} = V(I + \epsilon R)\Lambda V^{-1}$ from resonance coupling

The main departure from standard diagonal SSMs (S4D, DSS) is that we learn V rather than deriving it from the HiPPO framework. S4D fixes V based on polynomial approximation theory, which provides good inductive bias for certain tasks but limits adaptability. TEN's learned V trades theoretical guarantees for data-driven flexibility.

Whether this tradeoff is favorable depends on the task and dataset. On LRA, TEN and S4 perform comparably, suggesting that end-to-end learning recovers a basis of similar quality. On language modeling, TEN is slightly worse at short sequences but slightly better at long ones, possibly because the learned basis adapts to the statistics of natural language.

D Scaling Laws

We study how TEN perplexity scales with model size on WikiText-103 (seq len 2048).

Table 6: Perplexity vs. model size. All models trained for equal wall-clock time.

Model	Parameters	Perplexity	FLOPs/Token
TEN-Small	12M	24.3	0.8G
TEN-Base	42M	16.9	2.1G
TEN-Large	117M	14.2	5.8G
TEN-XL	324M	12.7	15.2G
Transformer-Base	44M	16.7	4.2G

TEN follows a smooth power-law scaling curve, similar to what has been observed for transformers and SSMs. The FLOP efficiency is better than transformers at all scales tested—TEN-Base achieves

comparable perplexity to Transformer-Base at roughly half the FLOPs per token. Whether this advantage holds at the 1B+ scale is unknown; we have not run those experiments.

E Sequence Length Generalization

We test whether TEN can generalize to longer sequences than it was trained on. This is relevant for applications where training on very long sequences is expensive but inference must handle them.

Table 7: Perplexity degradation when testing on sequences longer than training length.

Model	Train Len	Test Len	Degradation
Transformer	512	1024	+8.3
TEN	512	1024	+2.1
HTEN	512	1024	+1.4
Transformer	512	2048	+22.7
TEN	512	2048	+5.8
HTEN	512	2048	+3.9

TEN degrades much less than transformers when extrapolating to longer sequences. We attribute this to the continuous nature of eigenstate dynamics: the recurrence $c_k(t+1) = \lambda_k c_k(t) + \beta_k(t)$ does not depend on absolute position, unlike the learned positional embeddings used by the transformer baseline. Relative or rotary position encodings would likely close this gap for transformers, but we have not tested this.

F Connection to Attention

It is instructive to compare TEN and attention as matrix operations. Attention computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V \tag{35}$$

which involves a $T \times T$ attention matrix. TEN can be viewed as a factorized alternative:

$$\text{TEN}(X) \approx V \cdot \text{diag}(\Lambda^t) \cdot V^T X \tag{36}$$

where V (eigenvectors) plays the role of keys/queries and Λ^t (eigenvalue evolution) replaces the softmax weighting. The factorization through K-dimensional eigenstate space ($K \ll T$) is what reduces the cost from $O(T^2)$ to $O(TK)$.

This is not a formal equivalence—attention is content-dependent while TEN's eigenvalues are fixed for a given layer. But it helps build intuition for why TEN can capture similar patterns with lower cost: the eigenstate decomposition acts as a low-rank approximation of the attention pattern, with the rank K chosen to balance expressiveness and efficiency.

G Training Hyperparameters

For reproducibility, we list all hyperparameters:

- Optimizer: AdamW, $\beta_1 = 0.9$, $\beta_2 = 0.999$
- Learning rate: 3×10^{-4} with cosine decay
- Warmup: 2000 steps (linear)
- Batch size: 32 sequences
- Weight decay: 0.1
- Gradient clipping: max norm 1.0
- Dropout: 0.1 (feedforward layers only)
- Precision: bfloat16 mixed precision
- Hardware: 1× NVIDIA A100 80GB, CUDA 12.4, PyTorch 2.4
- Training time: 3 days for TEN-Base on WikiText-103

Eigenvectors are initialized with QR-orthonormalized Gaussian random matrices. Eigenvalue decay rates α_k are drawn from $\mathcal{U}(-3, 0)$; frequencies ω_k are initialized uniformly across $[0, 2\pi)$ and then learned. The resonance matrix is initialized as $R = I + 0.01M$ with $M \sim \mathcal{N}(0, 1/K)$.

We did not perform extensive hyperparameter search. The settings above worked well across all tasks; we suspect modest gains are possible with task-specific tuning, particularly of K and the eigenvalue initialization range.